

PROJECT DESCRIPTION

1. INTRODUCTION

Instead of learning big methodologies [...] focus on practices. [...] You can become an expert in a practice without being an expert in a complete methodology.

— Ivar Jacobson

This is a medium-sized **scientific software engineering project** with the following characteristics: it is to be pursued from **first principles**; it includes certain elements of **agile methodologies and DevOps**; it consists of a number of **activities** resulting in a number of **artifacts**; it treats **styling, versioning, hosting, testing, documenting, and communicating** as **primary concerns**, while **technologies and tools** as **secondary concerns**; and it involves **individual work** only.

It is common for **software for scientific computing** to carry out **numerical or symbolic calculations**. For example, a sophisticated, state-of-the-art, **scientific calculator** is such software. Furthermore, it is conceivable for a scientific calculator to have support for a number of things, including certain transcendental functions.

A **transcendental function** is a function that does not satisfy any single-variable polynomial equation whose coefficients are themselves roots of polynomials. For example, $\sin(x)$ is a transcendental function, but $\sqrt{x}$ is not.

You will be assigned a transcendental function pseudo-randomly at an appropriate time. This transcendental function will be from the following list:

- F1: $\arccos(x)$
- F2: $\tan(x)$
- F3: $\sinh(x)$
- F4: $\Gamma(x)$ (Gamma Function)
- F5: ab^x
- F6: $B(x, y)$ (Beta Function)
- F7: x^y
- F8: σ (Standard Deviation)

(F1 – F8 are identifiers for the sake of reference.) In the foregoing, x and y are real variables, and a and b are real constants, unless otherwise stated.

2. PROJECT

The work on the project has been divided into a **collection of related problems** and the problems have been grouped into a **collection of deliverables**, namely D1, D2, and D3.

3. PROCESS

The project progresses **nonlinearly (specifically, iteratively and incrementally)**. In particular, the problems in later deliverables depend on the problems in earlier deliverables.

In the rest of the document, “must” means mandatory and “encouraged” means optional.

4. CONSTRAINTS

The following constraints apply to each deliverable.

4.1. FILE FORMAT

Each deliverable must be typeset in LATEX.

4.2. GENERATIVE ARTIFICIAL INTELLIGENCE

Each problem of every deliverable must use one or more of publicly-available **generative artificial intelligence (GAI)** tools that rely on **large language models (LLMs)**. The prompts used must be based on the **CASTROFF Prompt Engineering Framework**. The **use as well as non-use** of these tools must be accompanied by **prompt types used, examples of prompts, and explanations of the output**.

4.3. ATTRIBUTION

All non-original work (that is, any work external to that team) in D1 must be **cited and referenced appropriately**. For example, ACM or IEEE citing and referencing style could be used. A comprehensive collection of resources on citing and referencing is available¹.

It is important that all expression in any form be **evidence-based**, and **not to make claims that cannot be substantiated**. Furthermore, it is important **not to copy others’ work verbatim** regardless of whether it is cited. A copy could be **syntactic or semantic**. A copied work does not receive any credit, as the credit goes to the original, not to its copy.

¹ URL: <http://library.concordia.ca/help/howto/citations.html> .

5. DELIVERABLE 1 (D1)

This involves preparing slides to be presented and demonstration to be given on Zoom.

The students are expected to have their audio and video on during their session.

PROBLEM 1. [20 MARKS]

Create a persona for a user of your function. There are multiple templates for creating a persona. Use a **mind map** to decide which template you should use.

PROBLEM 2. [30 MARKS]

Express requirements of your function based on one of the **styles** and **guidelines** given in the **ISO/IEC/IEEE 29148** Standard. Associate **each** requirement with a unique **identifier**. Make any assumptions **explicit**.

You are **encouraged** to use a tool for managing requirements, such as **Atlassian Jira**.

Note: Problem 2 must be informed by Problem 1.

PROBLEM 3. [20 MARKS]

List **two different algorithms** in an **established pseudocode format** for implementing your function. The algorithms must be **independent** of any programming language.

Note: Problem 3 must be informed by Problem 2.

PROBLEM 4. [30 MARKS]

Use a **mind map** to decide which algorithm you should use. **Implement your function** in Python using the selected algorithm. Your implementation **must** have a **textual user interface** for input and output.

Note: Problem 4 must be informed by Problem 3.

6. DELIVERABLE 2 (D2)

This involves preparing slides to be presented and demonstration to be given on Zoom.

The students are expected to have their audio and video on during their session.

PROBLEM 5. [60 MARKS]

Modify the implementation of your function in D1/Problem 4 to implement it from scratch in Python.

The term “scratch” means that, apart from the **functions related to input, output, arithmetic, and user interface design**, your implementation is **prohibited** from using any built-in or library functions provided by Python. This could lead to **identifying and implementing functions subordinate** to your given function.

Your implementation **must** have a **graphical user interface** (GUI) using **Tkinter**.

Your source code **must not** depend on any particular tool, including an integrated development environment (IDE), to compile.

Your source code **must** provide support for **handling exceptions**. This could be done either by using built-in exception classes provided by Python or writing your own exception classes.

Your **error messages must** aim to be **helpful** to users of your function.

PROBLEM 6. [20 MARKS]

Your Python source code **must** be placed in a public **distributed version control system**. You are **encouraged** to use a hosting service, such as **GitHub**. You must include the address of the hosting service you use.

Your **commit messages must** be of ‘high-quality’.

Your repository **must** have a **README** file.

PROBLEM 7. [20 MARKS]

Modify your requirements of your function in D1/Problem 2 based on D2/Problem 5. Give an updated list of your requirements.

7. DELIVERABLE 3 (D3)

This involves preparing a poster to be presented and demonstration to be given in-person, in the classroom.

PROBLEM 7. [70 MARKS]

Modify, as necessary, the implementation of your function in D2/Problem 5 to satisfy the following:

- Your Python source code **must** conform to an established **programming style** for Python, such as PEP-8.
- You **must** use a **tool**, such as **Flake8**, to check the conformance of your Python source code with PEP-8. You must include one or more **snapshots of the styling tool** you used to show its usage.
- You **must** use a **debugger**, such as **pdb**. You must include one or more **snapshots of the debugger** you used to show its usage.
- You **must** use a **static code analyzer**, such as **Pylint**. You must include one or more **snapshots of the static code analyzer** you used to show its usage.
- Your Python source code **must** use **Semantic Versioning**.
- Your GUI **must** use **user interface design principles (UIDP)**. There are multiple UIDP. Use a **mind map** to decide which UIDP apply and which do not.
- Your GUI **must** aim to be **accessible**.

PROBLEM 8. [30 MARKS]

Write unit test cases for your function.

You **must** use a **unit testing framework**, such as **PyUnit**.

You are **encouraged** to use a tool for managing test cases, such as **Atlassian Jira**.